

Análisis del código: identificadores, pseudoclasses, semántica y fluidez sin Media Queries

Explicación técnica basada en el HTML y CSS de la guía de Resident Evil 4.

1. Identificadores y selectores utilizados

A diferencia de un enfoque basado en etiquetas, aquí casi todo el estilo se aplica mediante **clases** (`class="..."`), mientras que los `id` (`#introduccion` , `#multimedia` , etc.) se reservan exclusivamente para los enlaces de ancla del menú de navegación, sin recibir estilos propios en el CSS. Esta separación es una buena práctica: el `id` identifica un destino único de navegación y la `class` controla la apariencia, reutilizable en varios elementos.

- Selectores de clase simples:** `.site-header` , `.content-section` , `.gallery-item` , `.character-card` — cada bloque visual tiene su propia clase, lo que permite reutilizar el mismo diseño en varias tarjetas (los tres `.character-card`) sin repetir código.
- Selectores descendientes:** `.site-nav a` , `.main-article p` , `.gallery-item img` , `.character-card h3` — combinan una clase "contenedora" con una etiqueta interna, de modo que, por ejemplo, solo los párrafos dentro de `.main-article` reciben justificación de texto, y no todos los párrafos del sitio.

2. Pseudoclasses empleadas

El código utiliza una única pseudoclase, pero clave para la interactividad del menú:

- `.site-nav a:hover` — pseudoclase de **estado**: cuando el cursor pasa sobre un enlace del menú, cambia su fondo y color de texto (`background-color: #b30000`), ofreciendo retroalimentación inmediata de que el elemento es interactivo, sin necesidad de JavaScript.

3. Justificación de las etiquetas semánticas

El HTML5 se apoya en etiquetas estructurales que describen el rol de cada bloque, no solo su apariencia:

- `<header>` y `<footer>` : delimitan la cabecera de presentación y el pie con derechos de autor, zonas que un lector de pantalla puede saltar u omitir fácilmente.
- `<nav>` : agrupa la lista de enlaces de ancla como un bloque de navegación identificable, distinto de un simple listado de contenido.
- `<main>` y `<section id="...">` : `main` marca el contenido central único de la página, y cada `section` con su propio `id` corresponde a un destino del menú (introducción, multimedia, galería, personajes), reflejando la estructura temática real del documento.
- `<article>` : se usa tanto para el texto principal (`.main-article`) como para cada ficha de personaje (`.character-card`), porque ambos son fragmentos de contenido que tendrían sentido de forma independiente (por ejemplo, si se extrajera la ficha de Ada Wong y se publicara aparte, seguiría siendo comprensible).
- `<figure>/<figcaption>` : igual que en el ejemplo anterior, asocia cada imagen de la galería con su descripción como una sola unidad.
- `<strong>` : resalta "Resident Evil 4" en la introducción, marcando importancia semántica real y no solo negrita visual.

4. Estrategia matemática/CSS para lograr fluidez sin Media Queries ni Flexbox/Grid

Este código resuelve dos problemas de responsividad clásicos con pura aritmética de porcentajes:

a) Video con relación de aspecto fija (truco del *padding-bottom*):

```

/* .video-container */
position: relative;
width: 100%;
padding-bottom: 56.25%; /* 9 / 16 = 0.5625 */
height: 0;

/* .video-container iframe */
position: absolute;
top: 0; left: 0;
width: 100%; height: 100%;

```

La clave matemática es que, en CSS, un `padding` expresado en porcentaje siempre se calcula sobre el **ancho** del contenedor, nunca sobre el alto. Al fijar `padding-bottom: 56.25%` (la proporción 9:16 de un video panorámico) sobre un elemento de altura `0` , el propio relleno "genera" una altura proporcional al ancho disponible. El `<iframe>` , posicionado en absoluto, se estira al 100% de ese espacio recién creado. El resultado es un video que siempre mantiene su relación de aspecto exacta, sea cual sea el ancho de pantalla, sin una sola *media query*.

b) Galería con `inline-block` y porcentajes:

```

/* .gallery-item */
width: 29%;
margin: 1.5%; /* 1.5% a cada lado */

```

El cálculo es: $29\% + 1.5\% + 1.5\% = 32\%$ por elemento. Con `display: inline-block` , los elementos fluyen horizontalmente como texto; tres tarjetas ocupan $32\% \times 3 = 96\%$ del ancho del contenedor, cabiendo siempre en una fila y acomodándose solas al reducirse el espacio (ayudadas por `min-width: 220px` , que evita que las tarjetas se compriman hasta ser ilegibles). Sumado a `max-width: 1100px; margin: 0 auto` en `.site-main` , todo el diseño escala de forma proporcional entre pantallas pequeñas y grandes sin recurrir a Flexbox, Grid ni consultas de medios.